

中山大學
SUN YAT-SEN UNIVERSITY

编译原理 |HW10

Java 与.NET 平台垃圾回收机制深度对比分析

学　　号: 23336179

姓　　名: 马福泉

专　　业: 计算机科学与技术

指 导 教 师: 李文军

起 始 日 期: 2026 年 5 月 24 日

结 束 日 期: 2026 年 5 月 24 日

实 验 地 点: B202

目录

1 引言 **2**

2 Java 平台 GC 机制深度剖析 **2**

2.1 堆内存布局与分代/分区模型 2
2.2 Card Table 与 Remembered Set 2
2.3 G1 GC：并发标记 + 分区逐出 3
2.4 ZGC：染色指针与全并发整理 3
2.5 Shenandoah 3

3 .NET 平台 GC 机制深度剖析 **3**

3.1 堆结构与分代模型 3
3.2 Card Table 与 Write Barrier 4
3.3 Workstation GC vs. Server GC 4
3.4 Background GC 与非阻塞栈扫描 5
3.5 堆段（Heap Segment）管理 5

4 多维对比分析 **5**

4.1 内存模型核心差异 5
4.2 并发安全保证机制 6
4.3 停顿时间与吞吐量量化对比 6
4.4 配置复杂度与可观测性 7

5 工程选型建议 **7**

6 总结 **8**

1 引言

垃圾回收的演进史本质上是停¨顿-吞吐这¨一对矛盾体的持续博弈史。Java 与.NET 作为两大主流托管平台，在 GC 设计哲学上呈现出殊¨途同归的¨态势——两者均从早期的分代复制/标记-清除出发，逐步走向并发标记、并发整理，最终目标均为亚毫秒级可预测停顿。然而，二者在内存抽象、并发安全保证机制、以及面向不同工作负载（桌面 vs. 服务端）的默认策略上存在显著差异。

本报告的行文逻辑如下：第 2 节深入 Java GC，重点覆盖 G1 与 ZGC 的 Region 架构、SATB/Load Barrier 并发标记与整理算法；第 3 节深入.NET GC，重点覆盖三代堆（Gen 0/1/2）、大对象堆（LOH）、钉住对象堆（POH）及 Server/Workstation 双模式；第 4 节进行多维对比；第 5 节给出工程选型建议。

2 Java 平台 GC 机制深度剖析

2.1 堆内存布局与分代/分区模型

现代 JVM 堆结构按收集器不同分为两类模型：

1. **传统分代模型**（Serial GC、Parallel GC）：堆固定划分为 **Eden**、两个 **Survivor** 区（S0/S1）构成的年轻代（Young Generation）以及老年代（Old Generation）。Eden 与 Survivor 比例为 8:1（可通过 `-XX:SurvivorRatio` 调节）。新生对象在 Eden 分配，经历 Minor GC 后存活对象晋升（Promote）至 Survivor 区，经多次 GC 仍存活后最终进入老年代。元空间（Metaspace，自 JDK 8 起替代永久代）位于本地内存（Native Memory），不归 GC 堆管理。

2. **Region 分区模型**（G1 GC、Shenandoah、ZGC）：堆被划分为等大小的 **Region** 逻辑单元。以 G1 为例，堆被分为约 2048 个 Region（可在 $1 \sim 512$ MB 区间内调优），每个 Region 在运行时动态承担 Eden、Survivor、Old 或 Humongous 角色。Humongous 对象（超过 Region 大小一半）跨越连续多个 Region 分配。GC 周期以 Region 为粒度进行选择（Ëvacuation）¨，挑选收益最高（垃圾最多）的若干 Old Region 与全部 Young Region 一并回收，此即 G1 的 **Mixed GC** 策略。

2.2 Card Table 与 Remembered Set

跨 Region 引用（Old→Young, Old→Old）的追踪是增量式回收的核心难题。Java 使用两层机制：

- **Card Table**：将堆映射为固定大小的 Card（通常 512 字节）。每当引用字段被写入，JIT 编译的代码在 Write Barrier 中将对应 Card 标记为脏¨（Dirty）。Minor GC 时只需扫描脏 Card 对应的内存区域，而非整个老年代。
- **G1 Remembered Set（RSet）**：每个 Region 维护一个 RSet，记录外部 Region 指向本 Region 内的引用。RSet 以哈希表组织，粒度可粗化（Coarsening）至 Region 级以控制内存开销。并发 Refinement 线程持续处理写屏障生成的脏¨ Card 日¨志并更新 RSet。

ZGC 与 Shenandoah 则放弃了繁重的 RSet 维护，转而依赖读屏障（Load Barrier）和染¨色指针（Colored Pointer）或 Brooks Forwarding Pointer 来实现并发移动——这是设计哲学上的一个根本分歧。

2.3 G1 GC：并发标记 + 分区逐出

G1（Garbage-First）自 JDK 9 起成为 JVM 默认 GC，其核心流程分为两个交替阶段：

1. **Young-only 阶段**：仅回收 Young Region。Eden 中存活对象被逐出（Evacuate）到 Survivor 或 Old Region。此阶段为 STW（Stop-The-World），但持续时间极短（通常 $<$ 10 ms）。
2. **Space Reclamation 阶段**：启动并发标记周期（Concurrent Marking Cycle），使用 **SATB（Snapshot-At-The-Beginning）**算法维护并发标记的正确性——在标记开始时对当前堆快照进行逻辑记录，Write Barrier 将修改前的旧引用值压入 SATB 队列供并发标记线程处理。标记完成后进入 Mixed GC 序列，每次同时回收 Young 及部分 Old Region。

G1 的停顿时间目标（Pause Time Goal）通过 `-XX:MaxGCPauseMillis` 设定（默认 200 ms），自适应地调节每次 Mixed GC 回收的 Old Region 数量以尽量逼近目标。

2.4 ZGC：染色指针与全并发整理

ZGC 自 JDK 11 引入，JDK 15 起标为 Production Ready。其核心设计是将对象元数据嵌入 64 位指针中，即 **染色指针（Colored Pointer）**：指针的高 4 位（Linux/x86_64 上为 bit 42–45，JDK 21 后因虚拟地址空间扩展有所调整）用于存储 `Finalizable`、`Remapped`、`Marked0`、`Marked1` 四个元数据位[1]。

每次 Load 操作均经由 **Load Barrier** 检查被引用对象的元数据状态。若发现对象尚未标记或尚未重映射（Remapped），读屏障当场修复引用并可能触发对象的并发移动。这一 Self-Healing 机制使得 ZGC 无需 STW 即可完成对象重定位——唯一短暂的 STW 出现在并发周期的起点（Root Scanning）和终点（处理 finalizable 对象），通常 $<$ 1 ms。

JDK 21 的 **分代 ZGC（Generational ZGC）**将堆划分为 Young 与 Old 两块逻辑空间。Young 对象分配频繁，回收也频繁（通过并发 Young GC）；Old 对象分配稀薄，回收压力小。这一策略显著降低了 ZGC 的内存开销（此前非分代版本需要预留更大堆空间以应对浮动垃圾）。

2.5 Shenandoah

Shenandoah 与 ZGC 同为全并发整理 GC，但并发移动实现不同：Shenandoah 使用 **Brooks Forwarding Pointer**（对象头部多一个指针字，指向对象的当前地址）。并发搬迁时，线程透过转发指针访问对象，搬迁完成后 CAS 更新转发指针。该方案的代价是每对象 8 字节的额外内存开销。

3 .NET 平台 GC 机制深度剖析

3.1 堆结构与分代模型

.NET CLR 的托管堆采用严格的三代模型，辅以两个特殊用途堆：

- **Gen 0（第 0 代）**：所有新分配的小对象（$<$ 85,000 字节）的起点。Gen 0 通常极小——Workstation 模式下约 256 KB，Server 模式下每个堆约 256 KB——设计目标是使其热数据常驻 L2/L3 缓存。Gen 0 GC 为 STW，频率最高，但通常 $<$ 1 ms。

[1] JDK 21 引入分代 ZGC（JEP 439）后，指针元数据的编码方式有调整，以支持 Young/Old 分代标记，但染色指针 + 读屏障的核心范式不变。

- **Gen 1（第 1 代）**：充当 Gen 0 与 Gen 2 之间的缓冲区。Gen 1 GC 发生时同时回收 Gen 0 与 Gen 1，存活对象晋升至 Gen 2。此设计避免 Gen 2 被过于频繁地触发。
- **Gen 2（第 2 代）**：承载所有长生命周期对象，规模最大。Gen 2 GC 为 Full GC，包含标记-计划-压缩（Mark-Plan-Compact）全流程，耗时最长但触发频率最低。
- **LOH（大对象堆，Large Object Heap）**：所有 $\geq 85{,}000$ 字节的对象直接分配至 LOH，不参与分代晋升。LOH 默认仅进行标记-清除（Mark-Sweep）而非压缩，以避免大对象拷贝的巨额开销。.NET Framework 4.5.1 起可通过 `GCSettings.LargeObjectHeapCompactionMode` 强制压缩。
- **POH（钉住对象堆，Pinned Object Heap）**：.NET 5 引入，专门存放需固定地址的对象（如异步 I/O 缓冲区、P/Invoke 传递的托管数组）。将钉住对象隔离于 Gen 0/1/2 之外，从根本上解决了长期困扰.NET 的钉¨ 住碎片化问¨ 题。

图 1 给出了.NET 托管堆的逻辑布局示意。

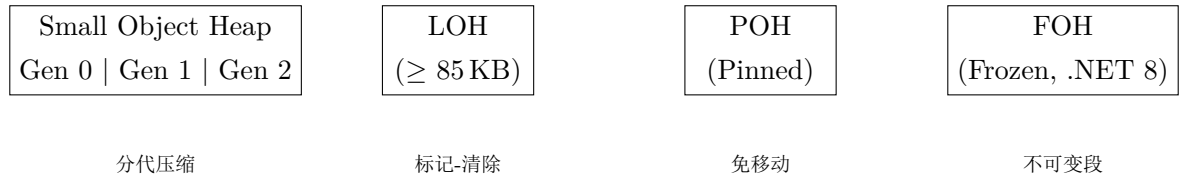


图 1: .NET 托管堆的逻辑分区（.NET 8+）

.NET 8 还增加了 **FOH（Frozen Object Heap）**，用于存储不可变对象（如字符串字面量、编译后的正则表达式等），其内存页被标记为只读，可被所有进程共享。

3.2 Card Table 与 Write Barrier

.NET 的跨代引用追踪同样依赖 Card Table。堆每 128 或 256 字节（取决于运行时实现）映射为一个 Card 位。JIT 编译器在每次引用写入操作（如 `a.field = b`）的机器码中插入 Write Barrier：若引用属于跨代（老对象引用新对象），则将对应的 Card 标记为已修改[2]。

Gen 0 GC 时只需扫描 Gen 1/Gen 2 中被标记的 Card 区域即可找到所有可能的跨代根引用，这被称为 "Card Table Scanning"。与 Java G1 的 RSet 相比，.NET 的 Card Table 更为简洁——无 RSet 维护的并发开销，但每次 Minor GC 需扫描的脏 Card 范围可能较大，在极端稀疏场景下效率稍逊。

3.3 Workstation GC vs. Server GC

CLR 根据宿主进程类型提供两种 GC 模式，由编译时配置或 `csproj/runtimeconfig.json` 控制：

[2]更精确地说，.NET JIT 在 x64 上使用一个位运算指令序列来标记 Card Table，确保屏障路径极短（通常 3–5 条指令）。

表 1: Workstation GC 与 Server GC 对比

属性	Workstation GC	Server GC
堆数量	1 个托管堆	N 个（N = 逻辑核心数）
并发 GC 线程	1 个专用后台线程	N 个线程（每堆 1 个）
Gen 2 后台 GC	支持（Background GC）	支持，各堆独立执行
分配缓冲区	单一分配上下文	每堆独立 Allocation Context
典型延迟	低（$<$ 5 ms Gen 0, $<$ 50 ms Gen 2）	稍高，但吞吐更优
适用场景	GUI/桌面应用	ASP.NET Core/云原生服务

Server GC 的关键优势在于 **每 CPU 堆（Per-CPU Heap）**设计：对象分配与回收均在各自 CPU 的本地堆上进行，消除了全局分配锁的竞争，且堆内对象天然具备良好的 NUMA 亲和性。Workstation GC 则保持单堆架构以最小化工作集，并优先追求低响应延迟（通过立即触发 GC 而非等待分配预算耗尽）。

3.4 Background GC 与非阻塞栈扫描

自.NET Framework 4.0 起，CLR 支持 **Background Gen 2 GC**：在后台线程并发执行 Gen 2 的标记和压缩阶段时，前台线程仍可分配对象并触发 Gen 0/Gen 1 回收（称为 Foreground GC）。这要求 Gen 0/1 的回收与后台 Gen 2 收集实现精细的互斥协调——.NET 使用记忆集（类似于 Java Card Table 但粒度更粗）确保后台收集器不会遗漏前台分配产生的新对象。

.NET 8/9 进一步引入了 **非阻塞栈扫描（Non-Blocking Stack Scanning）**：GC 不再强制所有托管线程在 GC 开始时立即进入安全点（Safe Point）并挂起；改为利用每个线程的 GC 内存栅栏（Memory Barrier），当线程返回到托管代码时主动检查并协作式挂起。这一改进将 Gen 0 停顿的 99th 百分位延迟降低了 30%–50%（据.NET 团队官方博客数据）。

3.5 堆段（Heap Segment）管理

.NET 堆由操作系统虚拟内存段（Segment）构成，每个段默认为 16 MB（Workstation）或 256 MB（Server，64 位）。段在物理上对应连续的虚拟地址范围（通过 `VirtualAlloc` 预留并 Commit）。.NET 8 引入 **Region-based Decommit（DATAS）**：堆不再以整个段为单位提交/解除提交，而是以 4 MB 的 Region 为单位精细管理物理内存页，大幅减少了长期运行服务的内存占用（Ḧeap Hard Limit 能̈ 力强化）。

4 多维对比分析

4.1 内存模型核心差异

表 2 汇总了两平台内存模型的关键架构差异。

表 2: Java 与.NET 内存模型对比

维度	Java (HotSpot JVM)	.NET (CLR)
堆基本单元	Region（G1/ZGC/Shenandoah）或连续代（Serial/Parallel）	连续代（Gen 0/1/2），Segment-level 管理
大对象处理	Humongous Region（跨连续 Region）	独立 LOH（≥ 85 KB），默认不压缩
跨区引用追踪	RSet（G1）/ 染色指针（ZGC）/ 转发指针（Shenandoah）	全局 Card Table（Write Barrier 标记）
钉住（Pinning）	不支持（对象不可固定，依赖 JNI 临界区）	原生支持，POH 隔离钉住对象
元数据存储	Metaspace（Native Memory）	托管堆内（MethodTable, SyncBlock 等与对象头耦合）

4.2 并发安全保证机制

两种平台保障并发 GC 正确性的核心机制截然不同，形成了写¨ 侧屏障与¨ 读¨ 侧屏障两¨ 个流派：

- **Java G1（SATB 写屏障）**：在写操作前将旧引用值记录至 SATB 队列，由并发标记线程异步消费。优点在于写屏障逻辑简单、执行快；代价是需要追踪所有标记开始前的逻¨ 辑快照，¨ 可能保留已死对象（浮动垃圾），增大内存压力。
- **Java ZGC（染色指针 + 读屏障）**：将 GC 状态嵌入指针本身，每次对象访问均通过读屏障检查状态——这被称为推¨ 迟到访问时刻的 GC 工作。¨ 读屏障频率远高于写屏障，但现代 CPU 的分支预测 + ZGC 的自愈（Self-Healing）属性将额外开销控制在 2%–5%（据 Oracle 2019 年 SPECjbb 基准测试）。
- **.NET（写屏障 + Card Table + 安全点协作）**：承袭经典的分代写屏障方案，无需读屏障（因.NET 不支持并发压缩搬迁，LOH 采用标记-清除），维持了极低的访问开销。代价是并发搬迁能力的缺失——Gen 2 压缩为 STW。

4.3 停顿时间与吞吐量量化对比

表 3 给出各主流 GC 配置在典型大堆（> 16 GB）场景下的名义性能特征（综合自各厂商官方文档及社区基准测试，非本报告实测数据）。

表 3: Java / .NET GC 停顿与吞吐特征对比（大堆场景，> 16 GB）

GC 实现	典型 Gen 0 / Minor 停顿	典型 Full GC 停顿	吞吐开销
Java G1	< 10 ms（Young STW）	< 200 ms（Mixed, 可调）	10–15%
Java ZGC (Gen)	< 1 ms（并发）	< 1 ms（并发）	5–15%
Java Shenandoah	< 1 ms（并发）	< 10 ms（并发）	10–20%
.NET Server GC	< 5 ms（Gen 0 STW）	50 ∼ 500 ms（Gen 2 STW）	5–10%
.NET Workstation	< 3 ms（Gen 0 STW）	30 ∼ 300 ms（Gen 2 STW）	5–8%

关键观察：

- ZGC 与 Shenandoah 在 Full GC 停顿上具有数量级优势，尤其适用于要求 < 1 ms 99th 百分位延迟的金融交易、实时竞价等系统。
- .NET Server GC（.NET 8+）的 Gen 0 停顿通常控制在 5 ms 内，且 Gen 2 停顿因后台并发 GC 对前台几乎透明。对绝大多数业务应用（$p99 < 200$ ms），此性能已充裕。
- 吞吐开销方面，.NET 的写屏障方案略优于 Java 的读写屏障方案，但差距不大（均在 5%–15% 范围）。

4.4 配置复杂度与可观测性

- **Java**：GC 调优参数繁多（> 50 个），涉及 Region 大小、IHOP（Initiating Heap Occupancy Percent）、并发线程数、NUMA 感知等。JDK 11+ 提供的 `JFR`（`JDK Flight Recorder`）和 `-Xlog:gc` 统一日志框架大幅提升了可观测性。
- **.NET**：GC 配置相对简洁（Latency Mode：Interactive/SustainedLowLatency/Batch + Heap Count 等少数参数）。诊断通过 `ETW`（`Event Tracing for Windows`） / `dotnet-counters` / `dotnet-gcdump` 完成。整体更偏向零¨ 配置优先生效的¨ 哲学。

5 工程选型建议

基于前述对比，给出面向不同负载特征的选型决策矩阵：

- **会话型服务（HTTP API、gRPC）**：典型对象生命周期短（请求-响应），Gen 0/1 回收收益极高。两者皆胜任；.NET Server GC 因 Per-CPU 堆在吞吐上略有优势，Java G1 因自适应停顿控制在大堆 > 32 GB 时更为从容。
- **低延迟交易/实时系统（$p99.9 < 1$ ms）**：必须 Java ZGC（JDK 21+ 分代版）或 Shenandoah。.NET 的 Gen 2 压缩 STW 在当前版本仍不可避免，难以满足该延迟等级。
- **桌面/GUI 应用**：.NET Workstation GC 为原生适配——单堆、低工作集、后台 Gen 2。Java 对应场景可考虑 Serial GC 或 G1。
- **大数据/分析型（MapReduce 风格、大堆批处理）**：优先吞吐。Java Parallel GC / .NET Server GC（Batch Latency Mode）均为合理选择。
- **钉住密集场景（P/Invoke、IO 密集.NET 服务）**：.NET POH 架构处理钉住对象天然优于 Java（Java 依赖 `GetPrimitiveArrayCritical` 等受限 JNI 机制，且可能阻止 GC）。

表 4 给出了推荐的快速映射。

表 4: 负载类型 → GC 选型快速映射

负载特征	Java 推荐	.NET 推荐
高吞吐 / 批处理	Parallel GC	Server GC (Batch)
通用 Web 服务	G1（默认）	Server GC
低延迟 / 实时	ZGC (Gen)、Shenandoah	（不推荐）
桌面 / GUI	Serial GC / G1	Workstation GC
IO 密集 / Pinning	避免大量 Pin（设计规避）	Server GC + POH

6 总结

本报告以内存布局、写/读屏障、并发整理算法及工程配置四个锚点，对 Java 与.NET 的 GC 体系进行了多层次对比。核心结论可归纳为三条：

1. **架构同源，实现分岔**：两者均源自分代回收理论，但 Java 在并发收集上走得更远——ZGC 与 Shenandoah 的读屏障 + 全并发整理实现了亚毫秒级 Full GC，而.NET 目前保留 Gen 2 的 STW 压缩，以换取更低的访问开销和更简洁的屏障实现。

2. **堆组织哲学的差异**：Java 以 Region 为统一抽象单元，实现了角¨ 色可自适应的¨ 灵活堆布局；.NET 则严格区分 SOH/LOH/POH/FOH 四种堆类型，对特定内存模式（大对象、钉住对象）提供专用优化路径。

3. **零配置 vs. 精细化调优**：.NET GC 的设计倾向开¨ 箱即用、默认高效，¨ 提供极少的调优旋钮；Java GC 提供丰富的调优参数，赋予有经验的开发者对 GC 行为的精细控制力。

随着云原生与 Serverless 的普及，容器化内存限制（Memory Cgroup）、毫秒级弹性扩缩容等需求正推动两者持续演进——Java 方向为栈上分配（Project Lilliput / Valhalla 值类型），.NET 方向正通过 DATAS（Region Decommit）与非阻塞栈扫描缩小与 Java 在停顿可控性上的差距。